==========

PlayList Name : CORE JAVA FULL COURSE

Channel: India Free Internship

Live Notes with Live coding

NotesLink:
https://github.com/indiafreeinternship/NOTES/tree/main/core%20java%20notes

==========

SESSION 1

Full Stack Java Developer

Module 1-: Programming Module (CoreJava, AdvJava, Framework-Spring)

Module 2-: UI Module (Html/CSS/JS/Angular/React)

Module 3-: DataBase Module (Oracle,MySql)

Module 4-: Testing Module (Testing basics: Selenium)

Module 5-: Tools Module (DevSecOps: Development Security and Operations tools)

JAVA

=====

PART 1: Core Java

PART 2: AdvJava

==========

CORE JAVA

==========

Language

========

=> Every language will have their own Alphabets.

=> Every lauguage will have their own Grammer.

=> Every language will have their own Constructions rules.

part 1: CoreJava

==========

1. Java Programming Components(java, Alphabets)

2. Java Programming concepts

3. Object Oriented Programming features.

==========

1. Java Programming Components(java, Alphabets)

(a) Variables

(b)Methods(Functions)

(c) Blocks

(d)Constructor

(e)classes

(f)interfaces

(g) AbstractClasses

2. Java Programming concepts

(a)Object-Oritented progrmming concepts

(b) Exception Handling process

(c) Multi-Threading process

(d) Java Collection Framework(JCF)

(e) File Storage

(f) Networking in java(Socket programming)

3. Object Oriented Programming features.

(a) Class

(b) Object

(c) Abstraction

(d) Encapsulation

(e) PolyMorphism

(f) Inheritance

Define StandAlone Applications :-

=>The application which are installed in one computer and performs actions in the same computer are known as Stand Alone application or DeskTop Application or Window applications.

=> According to developer StandAlone application means,

No Html Input

No Server Envrironment

No Database Storage

==========

AdvJava

==========

=> Advjava provide the following technolgies to construct web application.

(I)JDBC

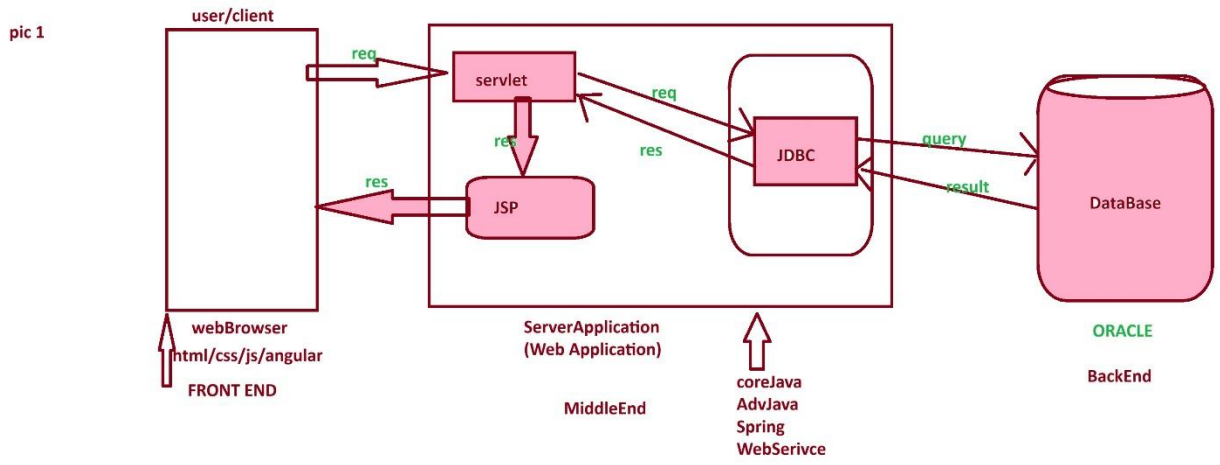
(ii)Servlet

(iii)JSP

Define Web Application

=>The Application which is executing in Web environment or internet environment is known as Web application or internet application.

Web Application Architecture pic 1:



==========

Servlet :- Servlet is a server program which accepts requets from user through WebBrowser.

JSP :- JSP stands for 'java Sever Page' and which is response from webApplication.

JDBC :- JDBC stands for 'java Database connectivity' and which is used to intract with Database.

==========

======

SESSION 2

======

What is the differenct b/w

(i) Language

(ii) Technology

(iii) Framework

(i) Language = Language provide programming components and concepts which are used in contructing program.

eg: java,python,c,c++

(ii) Technology = The process of Converting knowledge into realtime world application development is known as Technology.

eg: Login, Registration, AdvJava.

(iii) Framework = The Stucture which is ready constructed and available for application development is known as Framework.

eg : Spring, webServices

==========

Module 1-: Programming Module (CoreJava, AdvJava, Framework-Spring)

==========

Level -1: core java - standAlone Application

Level -2 : AdvJava - Web Application.

Level -3 : Framework - Enterprise application.

---

what is Enterprise application?

The application which are executing in distributed evironment and depending on the features like "Security", "Load balancing" and "clustring", are known as Enterprise Distributed Application or Enterprise Application.

==========

session 3

========

Define Program?

- Program is a set of instruction.

- After writing program, we have to save the program with language extension.

eg: Test.c

Test.cpp

Test.java

Stages of Program:

- The program will have following two stages:

1. Compilation

2. Execution

Compilation:

- The process of checking the program constructed according to the rule of language or not, is known as Compilation process.

- After Compilation process is Successfull, the sourceCode is converted into Compiled codes.

- C and C++ programs will generate ObjectiveCode and java program generate Byte code.

Execution

- The process of running Compiled codes and checking the required output is generated or not, is known as Execution process.

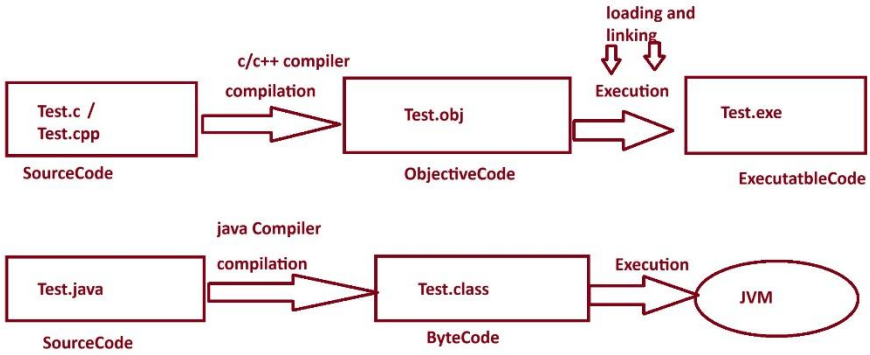
- In C and C++ languages, while executing Objective Code Internally " loading and Linking" process is performed and Object code converted into excutable code and generate result.

- In Java Language, the Byte code is executed directly on JVM (Java Virtual Machine)

==========

stages of program pic 2.

PIC 2



==========

session 4

========

what is the diff b/w

(i) Objective Code

(ii) Byte Code

(i) Objective Code

==========

=> The copiled code genreated from c and c++ program is known as Objective Code.

=> While Objective code generation OperatingSystem is participated, becuase of this reason ObjectiveCode is PlatForm dependent code.

DisAdvantages

=> Objective Code generated from one PlatForm cannot be executed on other PlatForm.

Note:

=> c and c++ language which are generating Objective Code are PlatForm dependent language.

(ii) Byte Code

==========

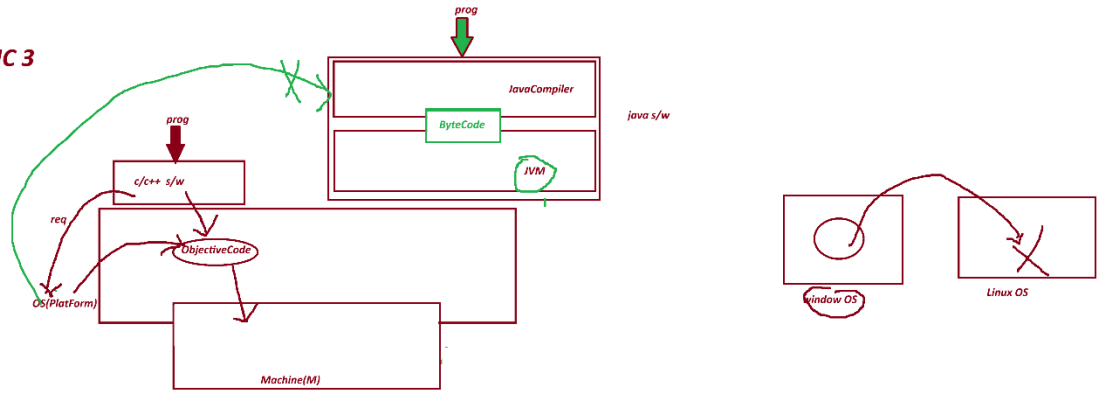
=> The byte code generated from one PlatForm canbe executed on all PlatForm based on JVM.

Note:

=> java language which is generating ByteCode is PlatForm independent language.

PIC 3

PIC 3



======

Define High Level language?

==========

=> The language format whic are understandable by the users are known as High Level Language.

c,c++,java

Define Low Level Language?

==========

=> The language format which are not understandable by the user are known as Low Level Language.

Ex: Machine languages.

Define Translator?

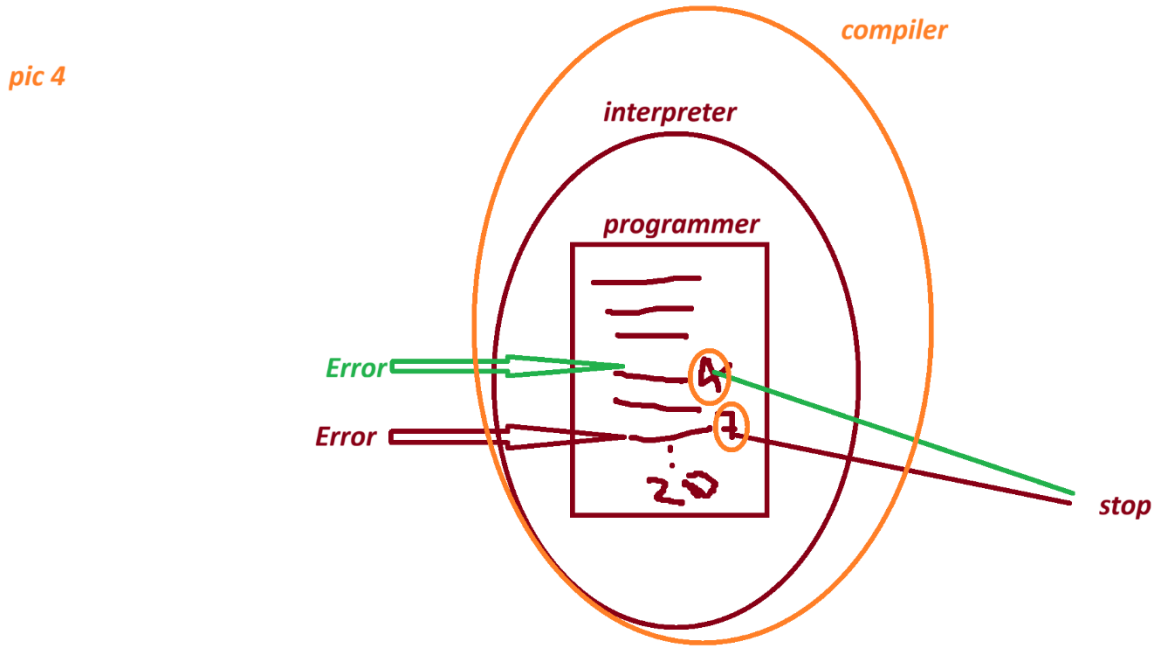
=> Translator are used to convert High Level language formats into Low Level Language formats and Low Level Language formats into High Level Language.

These Translators are categorized into two type.

(i) Interpreters - which translates program line-by-line.

(ii) Compiler - which translates total program at-a-time.

Pic 4



==========

1991 - "James Gosling" Sun Micro System - Code Write(programer)

WORA - Write Once and Run Anywhere.

Test.gt (Green Talk)

OAK

Java

==========

java versions:

1995 - java Alpha&Beta

1996- JDK 1.0

1997- JDK 1.1

1998 - JDK 1.2

2000 - JDK 1.3

2002 - JDK 1.4

2004 - Java5(Tiger)

2006 - java6

2011 - java7

2014- java8

2017 - java9

2018-java10 , java11

2019- java12,java13

..... 2026 -java 26

---

session 5

=======

What is the diff between ?

(i) JDK

(ii) JRE

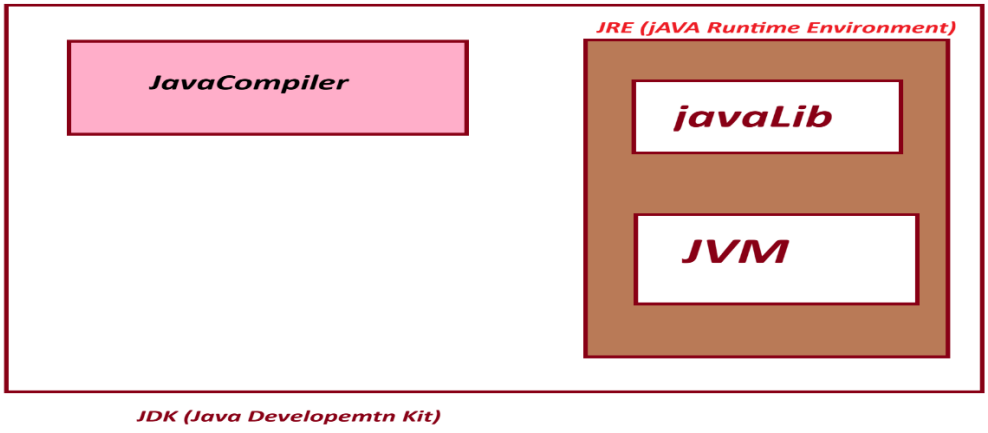
(iii) JVM

(i) JDK : JDK stands for "Java Development Kit" and which is the collection of the following:

(a) JavaCompiler

(b) JavaLib

(c) JVM



(a) JavaCompiler :- JavaCompiler will compile the source code and generate ByteCode when the compilation process is successfull.

(b) JavaLib :- JavaLib will provide pre-defined and ready constructed programming components which are used in construting application.

JavaLib= 'java'

CoreJava Packages:

(i) java.lang - Language package

(ii) java.util - Utility package

(iii) java.io - IO Stream and Files packages

(iv) java.net - Networking package

AdvJava packages:

(i) java.sql - DataBase connection package

(ii) javax.servlet - Servlet progrmming package

(iii) javax.servlet.jsp - JSP programming package

(c) JVM :- JVM Stands for "Java Virtual Machine" and which is used to execute JavaByteCode.

=> JVM internal partition of JDK.

=> The s/w components which intrnally having the behaviour like machine is known as Virtual Machine.

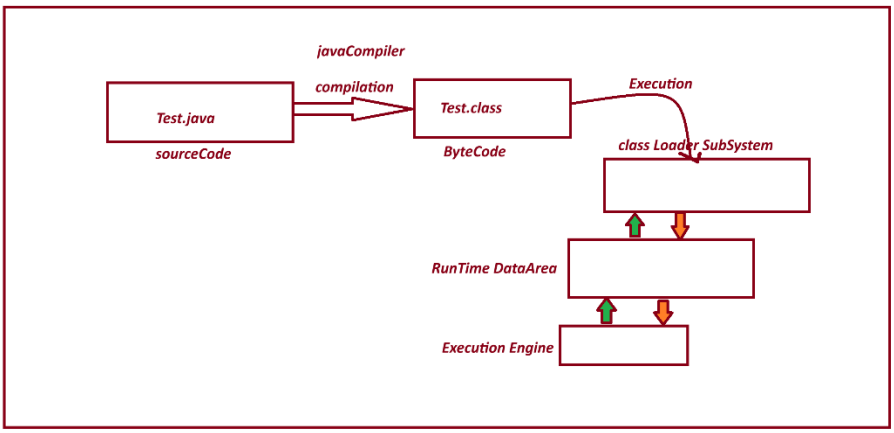
=> The Following are the parts of JVM:

1. Class Loader SubSystem

2. Runtime DataArea

3. Execution Engine

JVM PIC



==========

(ii) JRE :- JRE Stands for 'Java Runtime Environment' and which is collection JavaLib and JVM.

=> JRE is an internal partition of JDK.

========

session 6

========

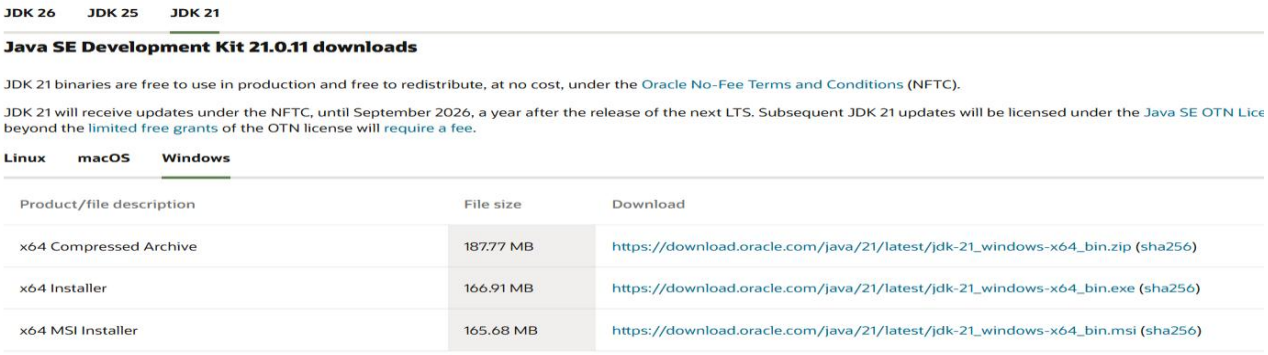
Installing Java s/w and setting path:

==========

step 1 :- Download java s/w(JDK) from Oracle Website.

Link: https://www.oracle.com/java/technologies/downloads/

step-2 : Install JDK-21



JDK 26 JDK 25 JDK 21

Java SE Development Kit 21.0.11 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN Lice beyond the limited free grants of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	187.77 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	166.91 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	165.68 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

Note: After Installation process we can find one folder with name "java" in program file.

C:\Program Files\Java

step 2 : set java path in "Envrironment variables"

click->new->

variables : path

variable : C:\Program Files\Java\jdk-21.0.11\bin;

Note:

=> Open CommandPrompt and check the follwoing commands are working or not:

javac - compilation commands

java -execution command

C:\Users\satru>java -version

java version "21.0.11" 2026-04-21 LTS

Java(TM) SE Runtime Environment (build 21.0.11+9-LTS-211)

Java HotSpot(TM) 64-Bit Server VM (build 21.0.11+9-LTS-211, mixed mode, sharing)

==========

session 7

========

Environment Variables :- Envrironment Variables are OperatingSystem variable, which hold the information about s/w components installed in Computer System.

These Environment variable are categorized into two types.

(i) User variable

(ii) System Variables

(i) User variable :- The variable whihc are related to individual users of Computer System are known as User Variables.

=>The information in User variable can be used by only individual user.

(ii) System Variables :-The variable which are related to all the users of computer system are known as system variables.

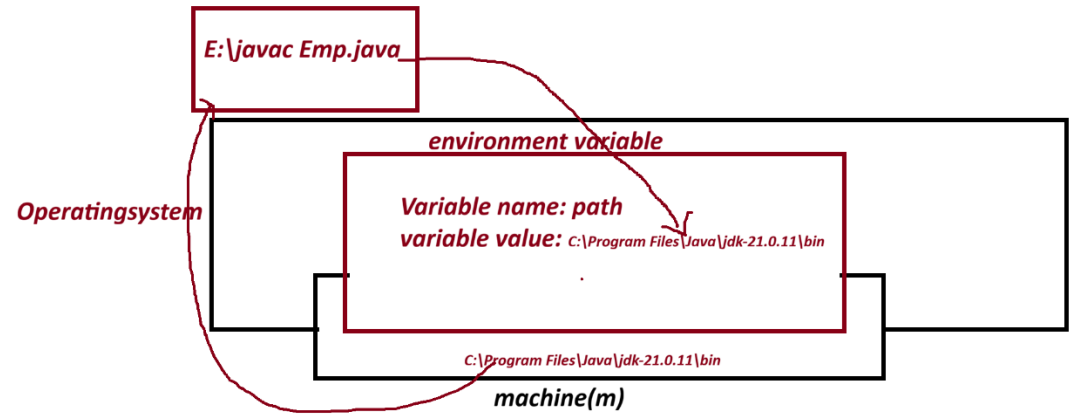
=> The Information in System variable can be used by all users of computer system.

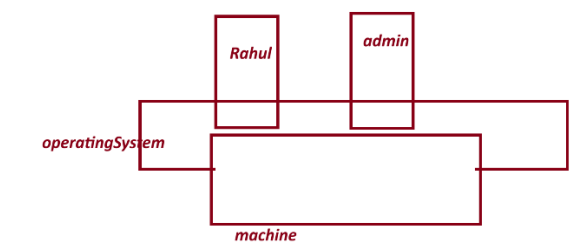
----

what is the advantage of having javaPath in environment variable?

=> When we have javaPath in Environment variable, then we can compile and execute java program from any location of computer System.

diagram:





session 8

define 'class'?

=> "class" is a "Structured Layout" generate "Object".

=> class in java is a collection of Variables, Methods, Blocks and Construtors.

=> The class in java can be added with main() to start the program execution.

=> main() is having the following pre-defined standard systax

ex: public static void main(String[] arg){}

=>we use "class" keyword in java to construct classes.

Structur of class in java:

```
class Class_name{
        //variable
        //methods
        //Blocks
        //Construtors
        //main()
}
```

==========

Ex program-1 : - Wap to display the msg as "Welcome to India Free Internship"?

```
class Display{
public static void main(String[] args){
 System.out.print("India Free Internship");
  }
}
```

step -1 Open Notedpad and type the program.

step -2 Save the program in folder with language-extendion (.java)

Systax :

Class_name.java

ex:

Display.java

step 3- File->Save-> Browser and select folder -> core java prgrams

-> save as type must be "All Files" -> click "save"

=> Open CommandPrompt and perform compilatin and Execution process

To open CommandPrompt , goto,folder-> type "cmd" in Addressbar and press "Enter"

step 4- compile the program as follows:

Syntax:

javac Class_name.java

ex:

javac Display.java

Step -5 Execute the program as follows:

Syntax:

java Class_name

Ex:

java Display

==========

Session 9

==========

Ex program -2

wap to display the sum of two numbers?

```
class Addition{
        public static void main(String[] args){
   int a=10;
```

```
    int b=20;
    int c=a+b;
    System.out.println("a value = "+a);
    System.out.println("b value = "+b);
    System.out.println("result (a+b) ="+c);
  }
}
```

ouput

=====

D:\core java programs>javac Addition.java (compilation)

D:\core java programs>java Addition (execution)

a value = 10

b value = 20

result (a+b) =30

==========

what is the diff b/w print() and println()

(i) print()

(ii)println()

(i)print():- print() method will display message and result, and makes the cursor wait in the same line.

(ii) println():- println() method also display message and result, and moves the cursor to next line or new line.

Note:- "+" symbol in print() method will concatenate(combine) message with result.

==========

Ex - program-3

wap to display Employee details?

(id,name,desg,sal)

Employee.java

```
class Employee
{
    public static void main(String[] args)
    {
      String id="SE121";
      String name="Alex";
      String desg="SE";
      int sal=30000;
      System.out.println("EmpId ="+id);
      System.out.println("EmpName ="+name);
      System.out.println("EmpDesg ="+desg);
      System.out.println("EmpSalary ="+sal);
    }
}
```

ouput

```
javac Employee.java
java Employee

EmpId =SE121
EmpName =Alex
EmpDesg =SE
EmpSalary =30000
```

==========

Assignment -1

wap to display BookDetails?

(code,name,author,price,qty)

Assignment -2

wap to display StudentDetails?

(name,branch,rollNo,6 sub marks, totMarks,per)

ouput

------

name=

branch=

rollNo=

s1=

s2=

s3=

s4=

s5=

s6=

totMarks=s1+s2+s3+s4+s5+s6;

per=

==========

session 10

=========

Naming Conventions in java.

=>The coding rules which are followed by the progrmmer in writing porgrams are known as Naming conventions in Java.

(Realtime Coding Standard).

package:

def => Packages are collection of Classes,interfaces and enum.

rule=> packages must be writing in Lowercase.

ex. com.app

==========

Classs and interfaces

def => Classes and Interface are collection of "variable and Methods"

rule : In Classes and interfaces, the starting letter of every word must be UpperCase.

ex: EmployeeSalary, DataInputStream

==========

Variables and Methods:

def : Variables are data holders and Methods are actions.

rule : In Variables and methods, the first word must be in LowerCase and from second onwords the starting letter must be UpperCase.

Ex: variable : rollNo, panCard,aadharCard

methods: readLine(), getSalary(),calculateMakrs(),.. etc

==========

Keywords:

def : The pre-defined built in words are known as keyword.

Rule : Keywords in java must be LowerCase.

Ex: void, static, public...

session 11

========

DataType:

Data Type specifies what type of data a variable can store in memory.

=> DataType in Java are categorized into two types.

1. Primitive DataType

2. Non-Primitive DataType

1. Primitive DataType:

-The "single valued data formats" are known as Primitive data types.]

-These Primitive data Type are categorized into four types.

(a) Integer DataTypes

(b) Float DataTypes

(c) Character DataType

(d) Boolean DataTypes

(a) Integer DataTypes

-The numrics data which is represented without decimal point are known as Integer DataTypes.

-These Integer dataTypes are categorized into four (4) types:

(i) bytes - 1 byte(8 bits)

(ii) short - 2 bytes

(iii) int - 4 bytes

(iv) long - 8 bytes

=> "byte" and "short" datatypes are used for Stream-data or MultiMedia data

=> "int" datatype is used for normal progrmming.

=> "long" datatype is used for biggest integer values like PhoneNo,AadharCardNo,BankCardNo....

=> In the process of assigning long-value we must use "L" or "l" in the RHS of declaration.

Ex: long phoneNo=99985241213L

==========

(b) Float DataTypes:

=> The numeric data with decimal point represented are known as Float DataTypes.

=> Float DataType are categorized into two types:

(i) float - 4 bytes

(ii) double -8 bytes

=> "float" datatype is used in normal programming.

=> "double" datatype is used to hold biggest scientific calculated values.

=> In the process of assigning float value we must use "F" or 'f' in the RHS of declaration.

Ex:

float f=123.34F

==========

ex : program

VariableDataType.java

```
class VariableDataType
{

   public static void main(String[] args){

     byte b=127;
     short s=1234;
     int i=1234567;
     long no=9898351245L;
     float f=123.5F;
```

```
    double d=23344.56;


    System.out.println("b value = "+b);
    System.out.println("s value = "+s);
    System.out.println("i value = "+i);
    System.out.println("no value = "+no);
    System.out.println("f value ="+f);
    System.out.println("d value = "+d);
  }
}
```

---

output

```
D:\core java programs>javac VariableDataType.java
D:\core java programs>java VariableDataType
b value = 127
s value = 1234
i value = 1234567
no value = 9898351245
f value =123.5
d value = 23344.56
```

==========

(c) Character DataType

- The "single valued Character" which is represented in single quotes is known as Character Datatype.

Ex: 'a', 'b', 'h', 'l', 'r' etc

char = 2 bytes

=========

why Character 2 bytes in java?

=> java language will use UNICODE (Universal Code) representation and which is ready to accept any type of language-codes available in the world.

=> The Folowing are some important language code in the world.

a. ASCII (Americal Standard Code for Information Interchange) for United State

b. ISO 8859-1 for Western European language.

c. KOI-8 for Russian.

d. GB18030 and BIG-5 for chines

==========

a. ASCII (Americal Standard Code for Information Interchange) for United State

=> The unique-numeric number which is genreated for every character entered from the keyboard is known as ASCII code.

==========

(d) Boolean DataTypes

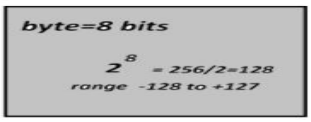
=> The datatype which is avalable in the form of true or false is known as Boolean DataType.

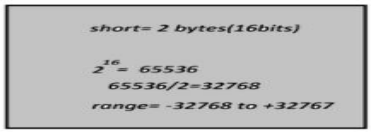
boolean =1 bits

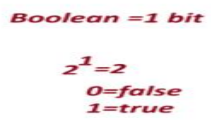
0=false

1=true

Diagram







session 12

==========

2. Non-Primitive DataType :-

The "group valued data formats" are known as Non-Primitive data Types or Referential datatypes.

- These Non-Primitive datatype are categorized into four types:

(a) Class

Ex class Student{

}

(b) Interface

Ex:

interfaces Animal{

}

(c) Array

Ex : int arr[]={10,20,40};

(d) Enum

enum Days{

MONDAY, TUESDAY

}

Note: "String is a Class in java and comes under Non-Primitive Datatype.

Ex String city="patna"

==========

Object Oriented Programming

=> The process of cosntructing programs using Class-Object concept is known as Object-Oritented Programming.

=> In Object oriented programming we work with Non-Primitive datatype or Referential datatypes.

=> The following are the levels in Object Oriented Programming.

1. Object definition
2. Object Creation
3. Object Location
4. Object components
5. Object Type.

6. Object Collection (Grouping object)

7. Object Sorting

8. Object Locking

9. Object Serialization

10. Object Cloning

==========

Define Object

=> Object is a Storage(memory) related to a class holding the member of class.

=> we use "new" keyword in java to create object.

Syntax :

Class_name obj_name=new Class_name();

==========

define 'class'?

=> "class" is a "Structured Layout" generate "Object".

=> class in java is a collection of Variables, Methods, Blocks and Construtors.

=> The class in java can be added with main() to start the program execution.

=> Classes in java are categorized into two types

(i) pre-defined class

(ii) user-defined class

(i) pre-defined class : The classes which are ready constructed and available from JavaLib are known as Pre-define classes or Built-in-classes.

Ex: String, System

(ii) user-defined class : The classes which are defined by the programmer are known as user defined classes.

Display

Addition

Employee

VariableDataType

==========

Variables in java:

=> variable are the data holder in the program

=> Based on Datatype the variable in java are categroiezed into two types:

1. Primitive DataType variables

2. Non-Primitive DataType variables

1. Primitive DataType variables : The variable which are declared with primitve data type like byte, short, int, long, double, char, boolean are known as primitve datatype variable.

=> These primitve data type variables will hold values.

2. Non-Primitive DataType variables : The variable which are declared with Non-primitve DataType like Class, Interface, Array, Enum are known as Non-Primitive DataType variables or referential data type variable.

=> These Non-Primitive Datatype variable will hold Object references.

==========

session 13

==========

define "static" ?

=> "static" keyword in java will decide the location of component memory in Class or Object.

=> bsed on "static" keyword the progrmming components are categorized into 2 types.

a. static programming components.

b. nonStatic programming components.

a. static programming components.

=> The programming components which are declared with "static" keyword are known as static programming components or Class Programming components.

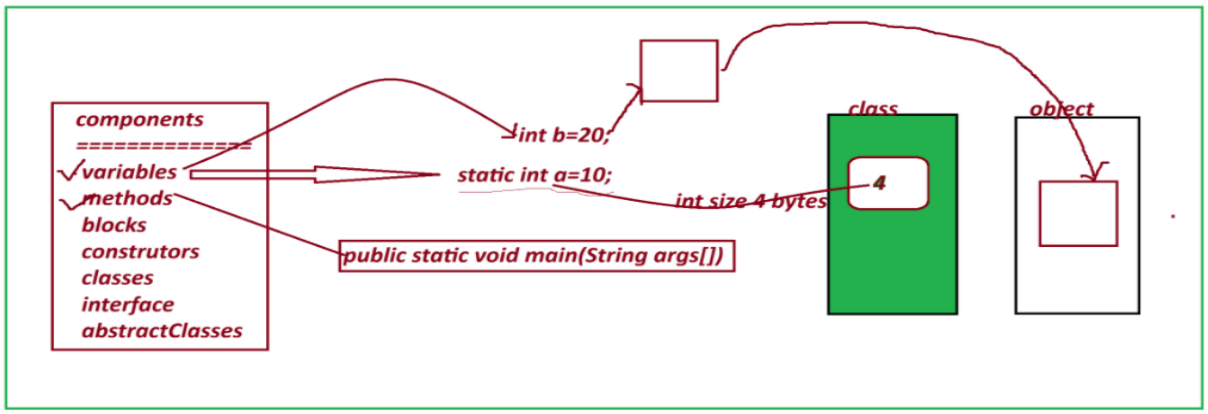
=> These static programming components will get the memory within the class and can be accessed with Class_name.

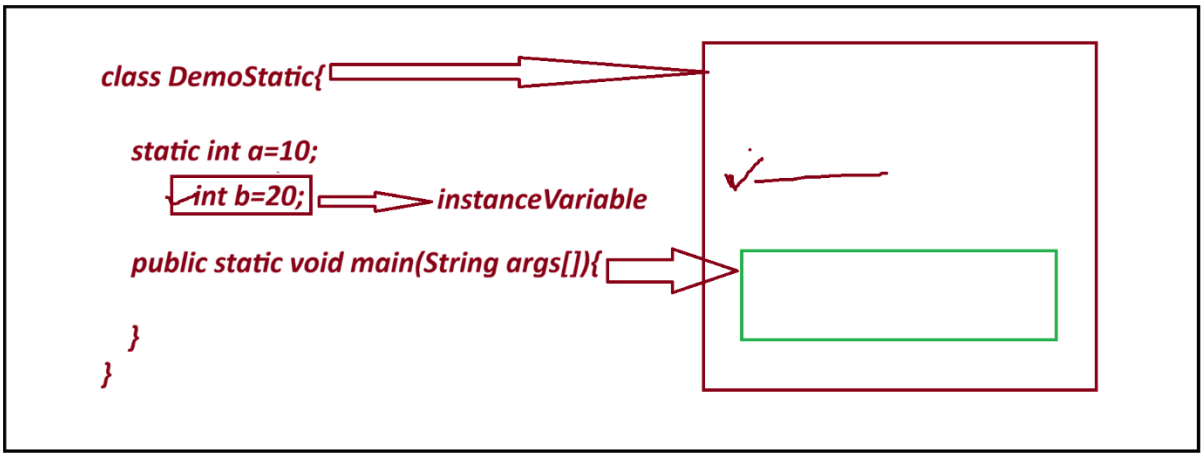
b. nonStatic programming components.

=> The programming components which are delcared without "static" keyword are known as NonStatic programming components.

=> These NonStatic programming components will get the memrory within the Object and can be accessed with Object_name.

==========





=> based on "static" keyword the variables are categorized into two types:

1. static variable

2. nonStatic variable

1. static variable:

=> The variable which are declared with "static" keyword are known as static variable or class variables.

=> These static variable will get the memory within the class while class loading and can be accessed with Class_name.

2. nonStatic variable

=> The variable which are declared without "static" keyword are known as NonStatic variables.

=> NonStatic variable are categorized into two types:

a. Instance Variables

b. Local Variables.

a. Instance Variables

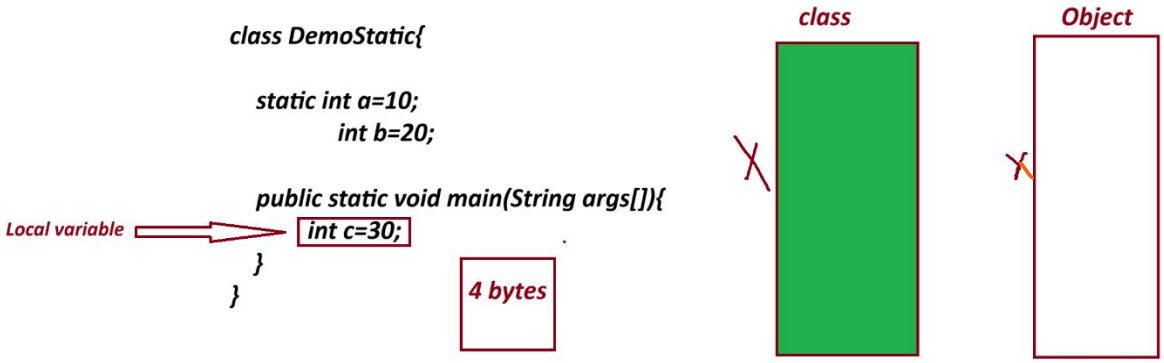
=> The NonStatic variable which are declared outside the method and inside class are known as instance Variables or Object_variable.

=> The instance variables will the get the memory within the object while Object creation process and can be accessed with Object_name.

b. Local Variables.

=> The NonStatic variable which are declared inside the methods are known as Local Variables or Method Variables.

=> These local variables will get the memory within the method while method exection and can be accessed directly inside the method.



==========

program :

DemoStatic.java

```
class DemoStatic{
        static int a=10;
          int b=20;
        public static void main(String args[]){
       int c=30;
     System.out.println("a value "+DemoStatic.a);
     DemoStatic ob=new DemoStatic();
     System.out.println("b value "+ob.b);
     System.out.println("c value "+c);
   }
}
```

output:

D:\core java programs>javac DemoStatic.java

D:\core java programs>java DemoStatic

a value 10

b value 20

c value 30

==========

session 14

==========

Method In Java

=> Methods are the actions which are performed to generate results.

=> based on "static" keyword the methods are categorized into two types:

1. static methods

2. NonStatic Methods (Instance methods)

1. static methods

=> The methods which are declared with "static" keyword are known as static methods or Class Method.

=> These static methods will get the memory within the class while class loading and can be accessed with Class_name.

structure of static methods:

```
static  return_type method_name(para_list)
{
        // method_body
}
```

=> These static methods are categorized into two types:

1. Pre-defined static methods

2. User defined static methods

1. Pre-defined static methods

=> The static methods which are constructed and available from javaLib are as known as Pre-defined static methods.

2. User defined static methods:

=> The static methods which are defined by the programmer are known as user defined static methods.

coding Rule:

=> static methods can access static variable of same class directly, but cannot access instance variable directly.

==========

ex program:

Method.java

```
class Method{
                    static int x=20;
              int y =30;
static  void m1()
{
          System.out.println("x value "+x);
      //System.out.println("y value "+y);
 }
public static void main(String[] arg){
   Method ob= new Method();
          ob.m1();
  }
}
```

output

```
D:\core java programs>javac Method.java
D:\core java programs>java Method
x value 20
```

==========

2. NonStatic Methods (Instance methods)

=> The methods which are declared without static keyword are known as NonStatic or Instance Methods.

=> These instance methods will get the memroy within the object while object creation process and can be accessed with Object_name.

Structure of Instance Method(NonStatic Methods)

```
return_type method_name(para_list)
{
        //method body
}
```

=> These Instance methods are categorized into two types:

1. Pre-defined Instance methods.
2. User defined Instance methods.

1. Pre-defined Instance methods.

=> The Instance methods which are constructed and available from javaLib are known as Pre-defined Instance methods.

2. User defined Instance methods.

=> The Instance methods which are defined by the programmer are known as user defined Instance methods.

coding Rule:

=> Instance methods can access both variables "static and Instance variable" directly, becuase the Object generated from classes and belongs to classes.

ex : program

Method.java

```
class Method{
        static int x;
          int y;

static  void m1()
```

```
{
        System.out.println("========static method m1()==========");
        System.out.println("x value "+x);

}
void m2()
 {
        System.out.println("======= non staic method m2()======");
        System.out.println("x value "+x);
        System.out.println("y value "+y);
 }
public static void main(String[] arg){
   Method ob=new Method();
   Method.m1();
  ob.m2();
  }
}
----------
output
D:\core java programs>java Method
========static method m1()==========
x value 0
======= non staic method m2()======
x value 0
y value 0
```

==========

==========

session 15

=========

JVM Architecture.

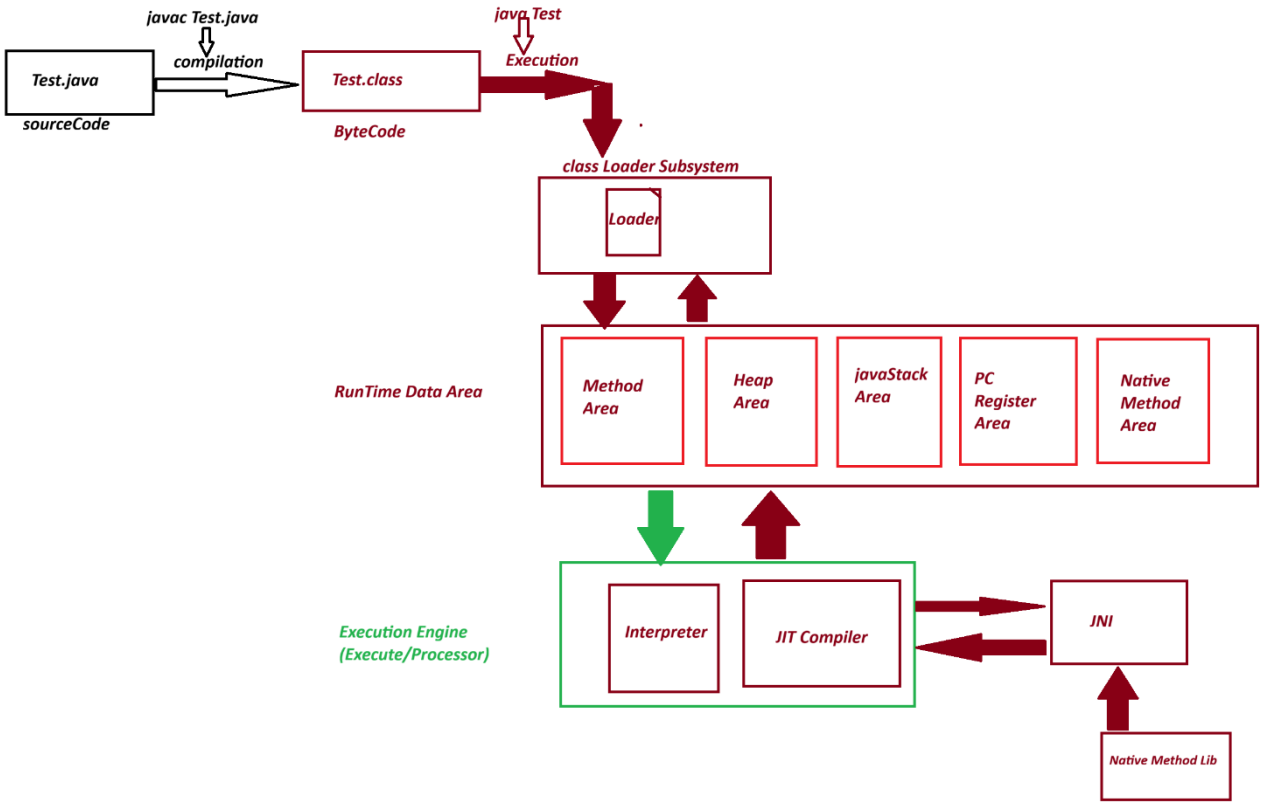
=> JVM Stands for Java Virtural Machine and which is used to execute javaByteCode.

=> This JVM Internally divided into the following three partition

1. Class Loader Subsystem

2. RunTime Data Area

3. Exectution Engine



---

1. Class Loader Subsystem

=> Class Loader Subsystem will load the class-file (byte-code) on to RunTime Data Area.

=> Class loadder Subsystem internally having loaders, linkers,verifiers and decoders.

2. RunTime Data Area

=> This Runtime data area internally divided into the following partition

1. Method Area

2. Heap Area

3. JavaStack Area

4. PC Register Area

5. Native Method Area

1. Method Area

=> This partition of Runtime data area where classes are loaded is known as Method Area.

=> while class loading static programming components will get the memory within the class.

=> In this process, main() method will get the memory within the class, and automatically copied onto JavaStackArea to start the execution-process

2. Heap Area

=> The partition of Runtime data Area where object are created is known as Heap Area.

=> while Object creation the Instance members will get the memory within the object

3. JavaStack Area

=> The partition of Runtime data Area where methods are executed is known as java Stack Area.

=> main() is the first method copied onto javaStackArea to start the execution process and main() will call remaining methods for execution.

4. PC Register Area

=> program Counter(PC) register will hold the status of method execution in java Stack Area.

=> Every method which is executing in javaStackArea will have its own program couner Register.

=> All these program counter registers are opened in a separated memory block known as PC register Area.

Advantage

======

=> Using the information in PC-Register the execution-engine will resume the execution process.

5. Native Method Area

=> The methods which are declred with "native" keyword in JavaLib are known as Native Methods.

=> These native methods internally having c/c++ codes.

=>when these native methods are used in application, then class loader subsystem will identify these methods and loads onto a separate memory block known as Native Method Area.

=>Execution-Engine will execute native methods using JNI(Java Native method Interface)

=> JNI while executing native methods used native method libraries.

Session 16

Define paramters?

=> Parameters are the variables which are used to transfer the data from one method to another method.

=> based on parameters the methods are categorized into two types:

(i) Methods without parameter

(ii) Methods with parameter.

(i) Methods without parameter

=> The methods which are declared without parameters are known as 0-Parameter methods without parameters.

(ii) Methods with parameter.

=> The methods which are declared with parameters are known as parameterized methods or Method parameters

example program

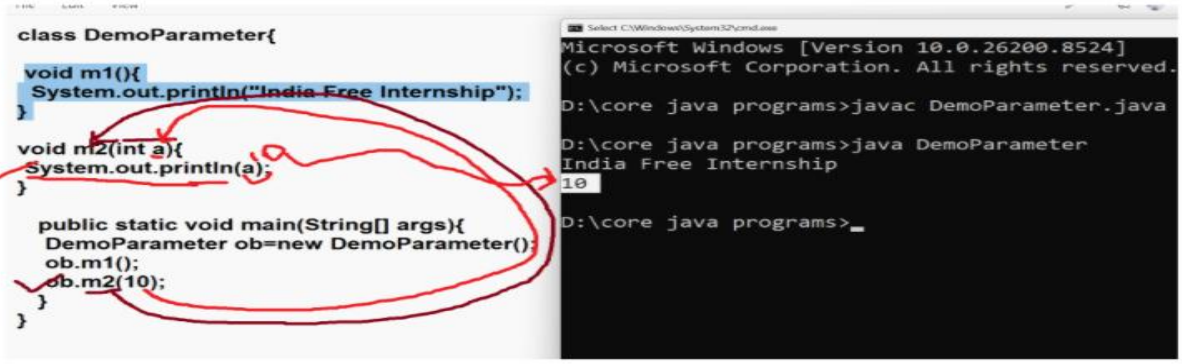
DemoParameter.java

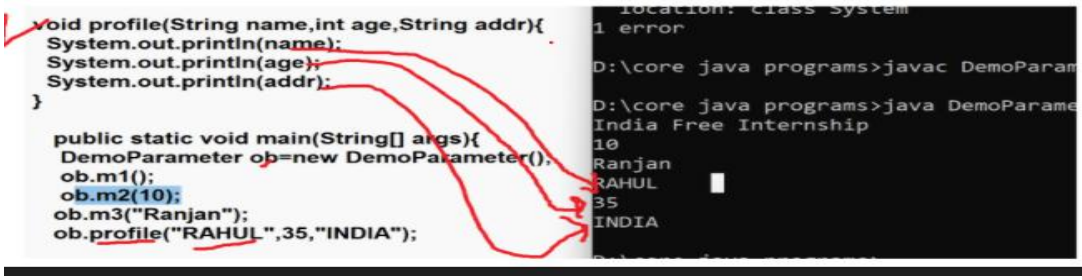
```
class DemoParameter{

 void m1(){
  System.out.println("India Free Internship");
}

void m2(int a){
 System.out.println(a);
}

void m3(String name){
        System.out.println(name);
}

void profile(String name,int age,String addr){
  System.out.println(name);
  System.out.println(age);
  System.out.println(addr);
}

   public static void main(String[] args){
    DemoParameter ob=new DemoParameter();
    ob.m1();
    ob.m2(10);
   ob.m3("Ranjan");
```

```
  ob.profile("RAHUL",35,"INDIA");


  }
}
```

output

```
--------
D:\core java programs>javac DemoParameter.java
D:\core java programs>java DemoParameter
India Free Internship
10
Ranjan
RAHUL
35
INDIA
```





Session 17

editPlus

download editPlus software

link : https://www.editplus.com/download.html

1. Return_Type

2. Scanner class

1. Return_Type

==========

=> "return_type" specify the methods will return the value after execution or not.

=> based on return_type the methods are categorized into two types:

1. Non return_type methods (void)

2. return_type methods (int, byte , short , long, class..)

1. Non return_type methods (void)

=> The methods which will not return, any value after method execution are known as Non return_type methods.

=> The methods which are declared with "void" are known as "Non return_type methods".

2. return_type methods

=> The methods which return the value after method exectution are known as return_type methos.

=> The methods which are declared without "void" are known as return_type methods.

example program

Demo.java

```
class Demo
{

        static int sum(int a,int b)
 {
       int c=a+b;
       return c;
 }

  static void mul(int x, int y){
              int z=x*y;

 }

       public static void main(String[] args){
              System.out.println("Session 17");
              int result=Demo.sum(10,20);
              System.out.println(result);

              //int mulResult=Demo.mul(2,3); error
              //System.out.println(mulResult);
```

```
    }
}
```

==========

output

Session 17

30

==========

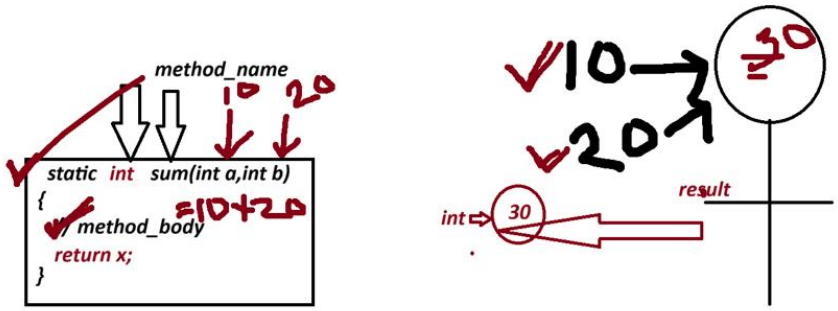
Scanner class

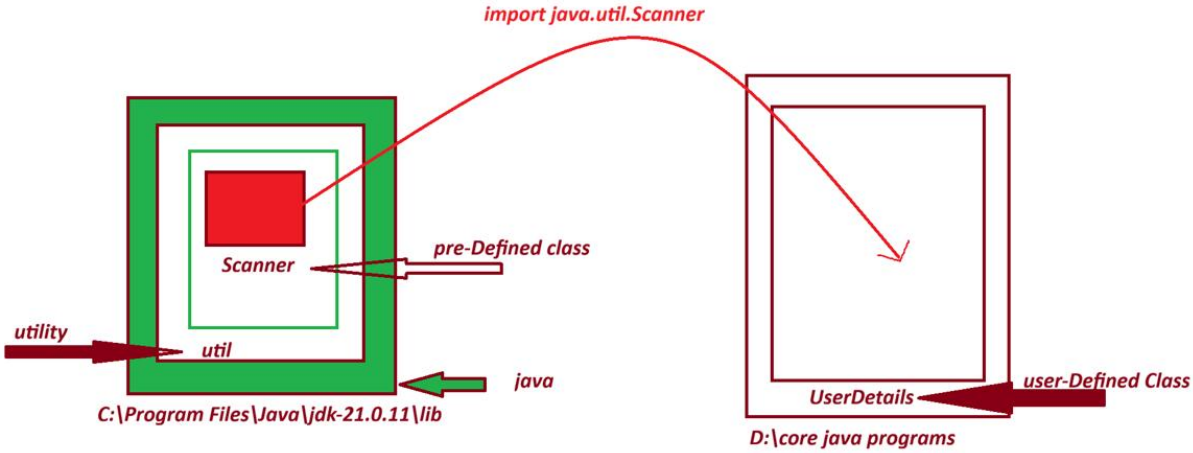
=> "Scanner" is a pre-defined class from "util"(Utility) package and which provide the following some important intance to read data into java program.

=>

1. **nextByte()**
2. **nextShort()**
3. **nextInt()**
4. **nextLong()**
5. **nextFloat()**
6. **nextDouble()**
7. **nextBoolean()**
8. **nextLine()**

==========





1. nextByte()=> used to read byte data

syntax:

byte val=s.nextByte();

2. nextShort() => used to read short data

syntax:

short var=s.nextShort();

3. nextInt() => used to read int data

syntax:

int var=s.nextInt();

4. nextLong() => used to read long data

syntax:

Long val=s.nextLong();

5. nextFloat()=> used to read float data

syntax:

float var=s.nextFloat();

6. nextDouble() => used to read double data

syntax:

double var=s.nextDouble();

7. nextBoolean()=> used to read boolean data

syntax:

boolean var=s.nextBoolean();

8. nextLine()=> used to read String data

syntax

String variableName=s.nextLine();

wap to read and display UserDetails?

(userName,mailId,phNo)

UserdDeatails.java

```
import java.util.Scanner;
class UserDetails
```

```
{


        public static void main(String[] args){
                Scanner s=new Scanner(System.in); // object Scanner
                System.out.println("Enter Username");
                String userName=s.nextLine();

                System.out.println("Enter mailId");
                String mailId=s.nextLine();

                System.out.println("Enter phone Number");
                Long phNo=s.nextLong();

                System.out.println("=======UserDetails==========");
                System.out.println("username :"+userName);
                System.out.println("mailId "+mailId);
                System.out.println("phone Number "+phNo);
        }
}
----------
output
D:\core java programs>javac UserDetails.java
D:\core java programs>java UserDetails
Enter Username
Rahul kumar
Enter mailId
```

indiafree123@gmail.com

Enter phone Number

7878123456

=======UserDetails==========

username :Rahul kumar

mailId indiafree123@gmail.com

phone Number 7878123456

Assignment 1

wap to read and display Book Details?

(name,author, price, qty)

Assignment 2:

wap to read and display Customer Details?

(custName,custCity,custMailid,custPhNo)

==========

session 18

==========

Assignment 1 (Solution)

---

wap to read and display Book Details?

(name,author, price, qty)

code

```
import java.util.Scanner;
class BookDetails
{
```

```
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the BookName: ");
                String name=s.nextLine();
                System.out.println("Enter the BookAuthor: ");
                String author=s.nextLine();
                System.out.println("Enter the BookPrice: ");
                int price=s.nextInt();
                System.out.println("Enter the bookQty");
                int qty=s.nextInt();
                System.out.println("==========BOOK DETAILS===========");
                System.out.println("BookName "+name);
                System.out.println("BookAuthor "+author);
                System.out.println("BookPrice "+price);
                System.out.println("BookQty "+qty);

        }
}
```

output

```
-------
Enter the BookName:
java-Language
Enter the BookAuthor:
B-Swamy
Enter the BookPrice:
400
```

Enter the bookQty

2

==========BOOK DETAILS==========

BookName java-Language

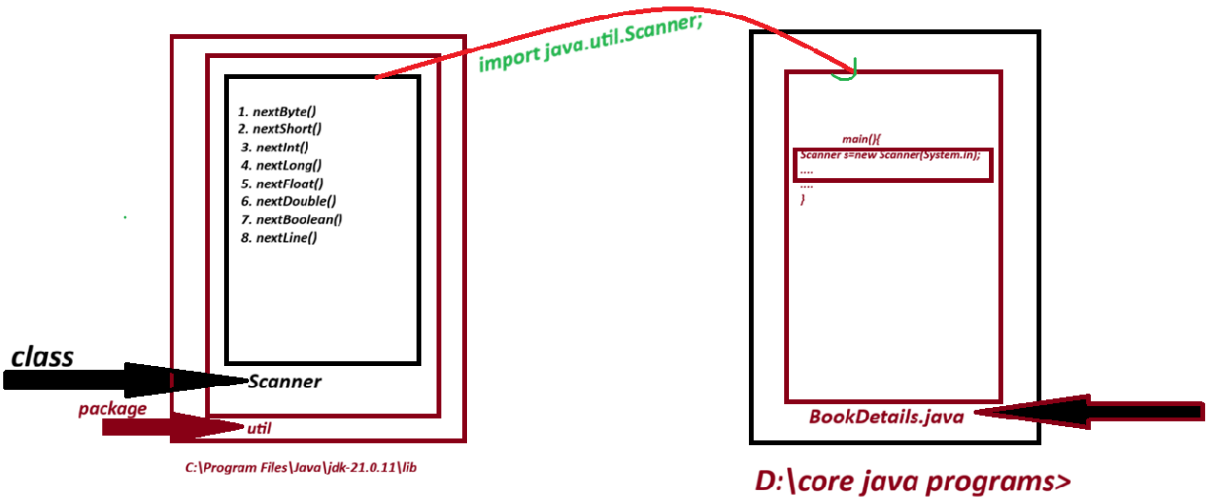
BookAuthor B-Swamy

BookPrice 400

BookQty 2

==========

Diagram



Operators In Java

==========

=> Operators are the special symbols which perform operations.

=> The following are some important operators used in java.

1. **Arithmetic Operators**
2. **Relational Operators**
3. **Logical Operators**
4. **Increment/Decrement Operators**

1. Arithmetic Operators

=> The Operators which perform basic operations or fundamental operations are known as Arithmetic Opertors.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	ModDivision

==========

2. Relational Operators

=> The Operators which are used to perform comaparions are known as Relational Operators.

Operator	Meaning
>	Greater Than
>=	Greater Than or equal
<	Less Than
<=	Less Than or equal
==	is equal
!=	Not equal to

==========

3. Logical Operators

=> The Operators which are used to compare two comparisions are known as Logical Operators.

Operator	Meaning
&&	Logical AND

|| Logical OR

! Logical NOT

==========

4. Increment/Decrement Operators

=> Increatment Operation will increament the value by-1 and decrement operatos will descrment the value by 1;

Operator	Meaning
++	Increment
--	Decrement

==========

Ex: wap to read 6 sub marks and display totMarks and Percentage?

```
import java.util.Scanner;
class StudentResult
{
	public static void main(String[] args)
	{
		Scanner s=new Scanner(System.in);
		System.out.println("Enter the marks of sub-1");
		int sub1=s.nextInt();
		System.out.println("Enter the marks of sub-2");
		int sub2=s.nextInt();
		System.out.println("Enter the marks of sub-3");
		int sub3=s.nextInt();
		System.out.println("Enter the marks of sub-4");
		int sub4=s.nextInt();
```

```
            System.out.println("Enter the marks of sub-5");
            int sub5=s.nextInt();
            System.out.println("Enter the marks of sub-6");
            int sub6=s.nextInt();

            int totMarks=sub1+sub2+sub3+sub4+sub5+sub6;
            float per=((float)totMarks/600)*100;

            System.out.println("==========StudentResult==========");
            System.out.println("Total Marks :"+totMarks);
            System.out.println("Percentage = "+per);
        }
}
```

output

```
=========
Enter the marks of sub-1
89
Enter the marks of sub-2
78
Enter the marks of sub-3
67
Enter the marks of sub-4
80
Enter the marks of sub-5
91
Enter the marks of sub-6
67
```

==========StudentResult==========

Total Marks :472

Percentage = 78.66667

========================

wap to read employee basicSal,hra,da and calculate totSal?

hra=91% of bSal

da=63% of bSal

totSal=bSal +hra+da

==========================

Control Structure in java.

====================

=> The Structures whicha are used by the programmer to control execution flow are known as Control Structures.

=> Control Structure in java are categorized into three types

1. Selection Statements

2. Iterative Statements

3. Branching Statements

1. Selection Statements

=> The Statement which are used to select one part of the program for execution are known as Selection Statements or Conditional Statement.

=> Types:

a. simple if

b. if-else

c. Nested if

d. Ladder if-else

e. switch-case

a. simple if:

=> simple if represents only true-block.

syntax:

```
if(condition){
//body
}
```

b. if-else:

=> if-else represents both true and false block.

syntax:

```
if(condition){
//body
}else{
    //body
}
```

c. Nested if :

=> The process of declaring "if" inside "if" is known as Nested-if or Inner-if.

syntax:

```
if(condition){
 if(condition){
  }
}
```

d. Ladder if-else

=> The process of interlinking more number of if-else blocks is known as ladder if-else

syntax:

```
if(condition){
        //body
}else if(condition){
        //body
}else if(condition){
        //body
}
.
.
.
```

e. switch-case:

=> switch-case statement is used to select one from multiple available options or cases.

syntax:

```
switch(value){
case 1: statements;
break;
case 2: statements;
break;
case 3: statements;
break;
...
..
..
case n: statements;
```

```
            break;
            default: statemetns;
            }
==============
session 19
==========
Ex-Program
----------
wap to read two integer values and display greater values?
(using subClass concept)
import java.util.Scanner;
class GreaterValue
{
    int greater(int x,int y){
        if(x>y){
            return x;
        }else{
            return y;
        }
    }
}

class DemoMethods1
{
    public static void main(String[] args)
    {
        Scanner s=new Scanner(System.in);
```

```
            System.out.println("Enter the int value-1:");
            int v1=s.nextInt();
            System.out.println("Enter the int value-2:");
            int v2=s.nextInt();
            GreaterValue gv=new GreaterValue();
            int res=gv.greater(v1,v2);
            System.out.println("Greater Value : "+res);
        }
}
```

---

output

Enter the int value-1:

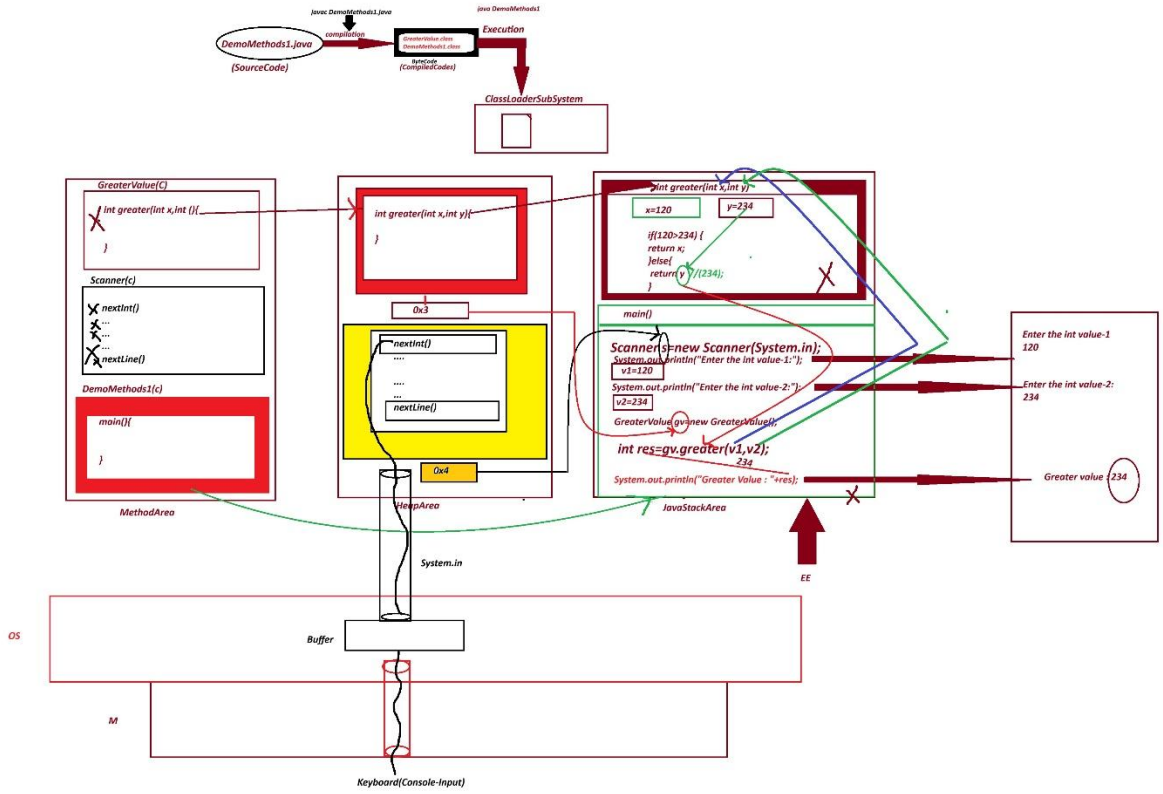
120

Enter the int value-2:

234

Greater Value : 234

Execution flow diagram



Assigment 1:

wap to read three integer values and display the value?

(using SubClass Concept)

Assignment 2:

Wap to read a value and display the value is Even or Not?

(using SubClass concept)

Even -display "true"

Odd- dispaly "false"

==========

Session 20